

SOLUTION ARCHITECTURE

Enterprise Aviation Data Platform (EADP)

Sprint 0 & Sprint 1 — Clear Step-by-Step Architecture

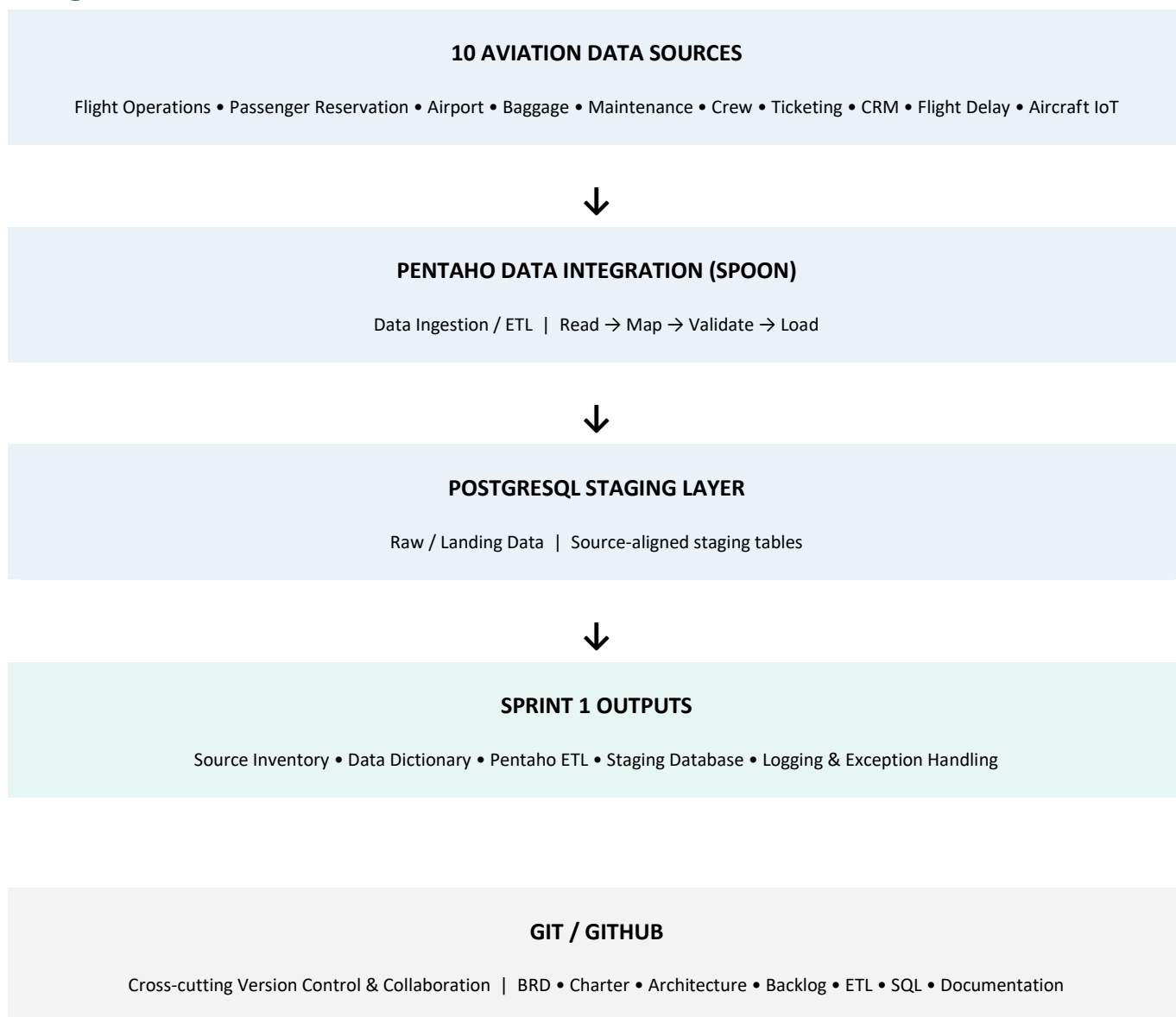
Prepared for: Enterprise Data Engineering Capstone Project

Current scope: Sprint 0 and Sprint 1

1. What is the Solution Architecture?

Solution Architecture describes how the different technical components of the Enterprise Aviation Data Platform work together to solve the business problem. For the current Sprint 0 and Sprint 1 scope, the main data flow is from the 10 aviation data sources, through Pentaho Data Integration, into PostgreSQL staging. Git/GitHub supports the project as the version-control and collaboration layer.

2. High-Level Architecture



3. Step 1 — Aviation Data Sources

The first layer contains the 10 datasets used in the project. These datasets represent different airline business systems and provide the raw input for the ingestion process.

#	Source System	Project Dataset / Format
1	Flight Operations System	Flight Operations System — CSV
2	Passenger Reservation System (PRS)	Passenger Reservation System — CSV
3	Airport Operations Database	Airport Operations — CSV
4	Baggage Handling System	Baggage Handling System — CSV
5	Aircraft Maintenance System	Aircraft Maintenance System — Excel
6	Crew Scheduling System	Crew_Scheduling_System — Excel
7	Ticketing & Booking System	Ticketing_Booking_System — Excel
8	Customer Relationship Management (CRM)	CRM — Excel
9	Flight Delay Reports	Flight_Delay_Dataset_70000_Rows — JSON
10	Aircraft IoT Sensor Logs	Aircraft_IoT_Sensor_Logs_80000_Rows — JSON

4. Step 2 — Pentaho Data Integration

Pentaho Data Integration (Spoon) is the ETL and data-ingestion component. It connects the source datasets to the PostgreSQL staging environment.

The basic Pentaho flow is:

1. Read the source file.
2. Map the source fields to the staging fields.
3. Perform basic data-type and structural validation.
4. Transform required fields into database-compatible values.
5. Route invalid or rejected records to an exception path.
6. Load successfully processed records into PostgreSQL staging.

5. Step 3 — Basic Validation and Exception Handling

Before loading records into PostgreSQL, the ingestion process should perform basic checks. Examples include checking required fields, data types, dates, numeric values and record structure. Records that cannot be processed should be captured through the exception-handling mechanism rather than silently discarded.

Check	Example	Expected Handling
Missing value	Aircraft_ID = NULL	Identify/log according to source rule
Wrong data type	Departure_Delay = ABC	Reject or route to exception
Invalid date	Unusable date value	Reject or transform according to rule
Malformed record	Incorrect source structure	Capture in error/exception path

6. Step 4 — PostgreSQL Staging Layer

PostgreSQL acts as the Raw/Landing Zone for Sprint 1. Successfully processed source records are loaded into source-aligned staging tables. This creates a common database environment while retaining the source-oriented structure.

Proposed staging-table naming convention:

- stg_flight_operations
- stg_passenger_reservation
- stg_airport_operations
- stg_baggage
- stg_aircraft_maintenance
- stg_crew
- stg_ticketing
- stg_crm
- stg_flight_delay
- stg_aircraft_iot

These are proposed names for architecture documentation; the assignment does not prescribe exact table names.

7. Step 5 — Sprint 1 Outputs

Deliverable	What it contains
Source Inventory	Source system, dataset/file, format, record count, key fields and business purpose.
Data Dictionary	Field names, data types, descriptions, source information and validation notes.
Pentaho ETL Pipelines	Transformations/jobs used to ingest the source datasets.
Staging Database	PostgreSQL staging tables containing successfully ingested records.
Logging & Exception Handling	ETL execution information, errors, rejected records and processing counts.
Git Commit History	Traceable commits for ETL, SQL and documentation changes.

8. Step 6 — Git / GitHub

Git/GitHub is not a data-processing stage. It is a supporting version-control and collaboration layer across the project. It stores the project artifacts and maintains the history of changes.

- BRD
- Project Charter
- Solution Architecture
- Sprint Backlog
- Pentaho transformation/job files
- SQL scripts
- Data Dictionary and Source Inventory
- Other project documentation
- Git commit history

9. Sprint 0 Architecture Responsibilities

Sprint 0 is the planning and architecture phase. The team should:

7. Understand the aviation business problem.
8. Identify stakeholders and their needs.
9. Identify and map the 10 data sources.
10. Define the project scope.
11. Prepare the BRD.
12. Prepare the Project Charter.
13. Design the high-level Solution Architecture.
14. Prepare the Product/Sprint Backlog.
15. Create the Git/GitHub repository and project structure.

10. Sprint 1 Architecture Responsibilities

Sprint 1 is the data discovery and ingestion phase. The team should:

16. Prepare the Source Inventory.
17. Prepare the Data Dictionary.
18. Develop Pentaho ingestion transformations.
19. Ingest the available CSV, Excel and JSON datasets.
20. Perform required basic validation and field mapping.
21. Create and populate PostgreSQL staging tables.
22. Implement logging and exception handling.
23. Maintain Git commits for the Sprint 1 work.

11. What is NOT part of the Current Sprint 0/1 Flow?

The following activities are not shown as current Sprint 0/Sprint 1 implementation stages: detailed Python/Pandas profiling, data-quality implementation, Star Schema design, enterprise data warehouse population, metadata, data lineage implementation and Power BI analytics. They belong to later phases of the project roadmap.

12. Final Architecture Explanation for Viva

Recommended explanation: "Our architecture starts with 10 heterogeneous aviation data sources. These sources are connected to Pentaho Data Integration, which acts as our ETL and data-ingestion layer. Pentaho reads the source files, maps and validates the data, handles exceptions, and loads successfully processed records into PostgreSQL staging tables. The staging layer acts as our raw landing zone for Sprint 1. Git and GitHub work as a cross-cutting version-control and collaboration layer where we maintain our ETL files, SQL scripts and project documentation. This architecture establishes the foundation for the later phases of the project."

13. Architecture Summary

The current architecture can be summarized as: 10 Aviation Data Sources → Pentaho Data Integration → PostgreSQL Staging / Raw Landing Zone. Git/GitHub supports the entire project through version control and collaboration.